

Audit Log System — Architecture & Implementation Guide

Project: A Night of Angels — event ticketing & back office **Stack:** Angular (static prerender + CSR admin) · Netlify Functions (`.mts`) · Netlify Blobs · self-hosted JWT auth **Status:** Design proposal — v1.0 **Audience:** Maintainers implementing the audit trail

1. Overview

An **audit log** is an append-only, tamper-evident record of *who did what, to what, when, and from where*. It answers questions like:

- Who checked in guest *X*, and at what time?
- Who deleted the 40 attendees that went missing last night?
- Which admin changed the SMS provider, or turned on maintenance mode?
- Who granted *user Y* the owner role?
- Were there failed login attempts against the owner account?

This document specifies a complete audit system that fits the project's existing architecture (Netlify Functions backed by Netlify Blobs, JWT-based roles) with **no new infrastructure**. It covers the data model, a race-free storage layout, the capture mechanism, an owner-only query API and admin UI, integrity/tamper-evidence, retention, and a phased implementation plan with copy-paste-ready code.

1.1 Goals

- **Accountability** — every mutating, security-relevant action is attributable to an actor.
- **Immutability** — records are append-only; the app never updates or deletes individual events in normal operation.
- **Non-intrusive capture** — logging never breaks or slows the primary action noticeably.
- **Queryable** — filter by date, actor, action, and entity; export for review.
- **Tamper-evident** — a hash chain makes silent edits/deletions detectable.
- **Zero new infra** — reuse Netlify Blobs and the existing auth model.

1.2 Non-goals

- Not a real-time SIEM or alerting pipeline (can be layered on later via log drains).
 - Not application performance tracing or debug logging.
 - Not a replacement for Netlify's platform/function logs.
 - Not full end-to-end encryption of the log (access is controlled by role + platform).
-

2. What we audit — event taxonomy

Audit **security-relevant** and **state-changing** actions. Pure reads are generally *not* logged, with two deliberate exceptions: bulk export of PII and viewing the audit log itself.

Domain	Action key	Trigger	Severity
Auth	auth.login.success	Successful sign-in	info
Auth	auth.login.failure	Bad credentials	warning
Auth	auth.logout	Sign-out	info
Auth	auth.password.reset_requested	Reset token issued	info
Auth	auth.password.reset_completed	Password changed via token	warning
Auth	auth.password.changed	Password changed while signed in	warning
Team	team.user.created	New back-office user	warning
Team	team.user.role_changed	Role updated	critical
Team	team.user.deactivated	User soft-deleted	critical
Team	team.user.reactivated	User restored	warning
Attendees	attendee.registered	New attendee	info
Attendees	attendee.checked_in	Door check-in	info
Attendees	attendee.checkin_overridden	Manual check-in toggle	warning
Attendees	attendee.table_assigned	Table set/cleared	info
Attendees	attendee.archived	Soft-delete	warning
Attendees	attendee.bulk_archived	Bulk soft-delete	warning
Attendees	attendee.restored	Un-archive	warning
Attendees	attendee.deleted	Permanent delete	critical
Attendees	attendee.bulk_deleted	Bulk permanent delete	critical
Attendees	attendee.exported	CSV/Excel/PDF export of PII	warning
Ticketing	ticket.emailed	Ticket sent by email	info
Ticketing	ticket.smsed	Ticket sent by SMS	info
Messaging	sms.broadcast_sent	Bulk SMS to guests	warning
Reservations	reservation.status_changed	Approve/decline/etc.	info
Settings	settings.updated	Any settings change	warning
Settings	settings.maintenance_toggled	Maintenance on/off	warning
Settings	settings.sms_provider_changed	Twilio ⇌ Termii	warning
Sponsors	sponsor.created / .updated / .deleted	Sponsor CRUD	info
Sponsors	sponsor.reordered / .visibility_changed	Order/enable toggle	info
Audit	audit.viewed	Someone opened the audit log	info
Audit	audit.exported	Audit log exported	warning

Severity drives UI emphasis and (optionally) retention. **critical** events should never be auto-pruned before the retention window and are candidates for future alerting.

3. Data model

A single, stable event shape across all domains.

```
// netlify/shared/audit-types.ts
export type AuditSeverity = 'info' | 'warning' | 'critical';

export interface AuditActor {
  id: string | null;      // user id (JWT sub); null for anonymous/system
  name: string | null;    // display name at time of action
  email: string | null;   // email at time of action
  role: string | null;    // role at time of action
}

export interface AuditTarget {
  type: string;           // 'attendee' | 'user' | 'settings' | 'sponsor' | ...
  id: string | null;      // primary key of the affected entity
  label: string | null;   // human label (e.g. attendee name, ticket code)
}

export interface AuditEvent {
  id: string;             // uuid v4
  seq: number;            // monotonic sequence (see §4.3)
  ts: string;             // ISO 8601 timestamp (server clock)
  action: string;         // taxonomy key, e.g. 'attendee.deleted'
  severity: AuditSeverity;
  outcome: 'success' | 'failure';
  actor: AuditActor;
  target: AuditTarget | null;
  // Field-level change set for updates: { field: { from, to } }
  changes?: Record<string, { from: unknown; to: unknown }>;
  // Small, non-sensitive extras (counts, reasons, provider names, etc.)
  metadata?: Record<string, unknown>;
  // Request context
  ip: string | null;
  userAgent: string | null;
  requestId: string | null;
  // Tamper-evidence (see §7)
  prevHash: string | null; // hash of the previous event in the chain
  hash: string;            // sha256 over the canonical event minus `hash`
}
```

3.1 Field rules

- **Never** store secrets, raw passwords, JWTs, API keys, full card data, or full message bodies. Store *derived* facts (e.g. `metadata.recipientCount: 48`, `metadata.channel: 'dnd'`).
- `changes` holds a **diff**, not full records, and redacts sensitive fields (see §8.2).
- `label` is a snapshot — it stays meaningful even after the target is deleted.
- Timestamps are **server-generated**; never trust client time.

4. Storage design on Netlify Blobs

Netlify Blobs is a key/value store with **no transactions** and a **last-write-wins** race on concurrent writes to the same key. The bulk-delete lost-update bug already fixed in this codebase is the cautionary tale. The audit log must therefore avoid any shared mutable key on the hot path.

4.1 One blob per event (append-only, race-free)

Write **each event to its own unique key**. Unique keys never collide, so concurrent writers never race.

```
Store: "audit"
Key:   "events/<YYYY>/<MM>/<DD>/<ts>-<uuid>.json"
Value: the AuditEvent JSON
```

Example: `events/2026/07/30/2026-07-30T18:04:12.481Z-6f1c...json`

Benefits:

- **No read-modify-write** → no lost updates, no contention.
- **Date-prefixed** → efficient range listing and retention by prefix.
- **Naturally ordered** — keys sort lexicographically by time within a day.

Trade-off: querying requires `list({ prefix })` + fetching matching blobs. At this project's scale (hundreds of guests, a handful of admins) volumes are small — thousands of events per event-season — so listing is inexpensive. §6.3 covers the pagination/cursor approach; §10 covers a rollup optimization if volume ever grows.

4.2 Store the actor context once per request

Extract the actor from the verified JWT (`sub` , `name` , `email` , `role`) and request headers (`x-forwarded-for` , `user-agent` , `x-nf-request-id`) inside the audit helper, so call sites stay terse.

4.3 Sequence + hash chain pointer

A tiny, low-contention pointer blob tracks the chain head:

```
Key: "chain/head"  ->  { seq: number, hash: string }
```

The head is read-modify-written **only from the audit writer**, which is already serialized per function invocation for a given action. Concurrency here is low (admin actions are not high-throughput). To stay correct under rare concurrent writes, the writer uses an **optimistic retry** (read head → compute → write with a compare; on mismatch, re-read and retry a few times). If you prefer to avoid the shared head entirely, drop `seq` /chain and rely on per-event hashes only (weaker tamper-evidence — see §7.3).

5. Capture mechanism

A single shared helper keeps call sites to one line and guarantees consistent shape, redaction, and error isolation.

5.1 netlify/shared/audit.ts

```
import { getStore } from '@netlify/blobs';
import { createHash, randomUUID } from 'node:crypto';
import type { TokenPayload } from './auth';
import type { AuditEvent, AuditActor, AuditTarget, AuditSeverity } from './audit-types';

const STORE = 'audit';

function store() {
  return getStore({ name: STORE, consistency: 'strong' });
}

function pad(n: number): string {
  return String(n).padStart(2, '0');
}

function keyFor(ts: Date, id: string): string {
  const y = ts.getUTCFullYear();
  const m = pad(ts.getUTCMonth() + 1);
  const d = pad(ts.getUTCDate());
  return `events/${y}/${m}/${d}/${ts.toISOString()}-${id}.json`;
}

function actorFrom(payload: TokenPayload | null): AuditActor {
  if (!payload) return { id: null, name: null, email: null, role: null };
  return { id: payload.sub, name: payload.name, email: payload.email, role: payload.role };
}

function requestContext(req: Request) {
  return {
    ip: (req.headers.get('x-forwarded-for') || '').split(',')[0].trim() || null,
    userAgent: req.headers.get('user-agent'),
    requestId: req.headers.get('x-nf-request-id'),
  };
}

// Canonical JSON (stable key order) so hashes are reproducible.
function canonical(value: unknown): string {
  if (value === null || typeof value !== 'object') return JSON.stringify(value);
  if (Array.isArray(value)) return `[${value.map(canonical).join(',')}]`;
  const obj = value as Record<string, unknown>;
  const keys = Object.keys(obj).sort();
  return `{${keys.map((k) => `${JSON.stringify(k)}:${canonical(obj[k])}`).join(',')}`}`;
}

function sha256(s: string): string {
  return createHash('sha256').update(s).digest('hex');
}

interface HeadRecord { seq: number; hash: string }

async function advanceChain(eventWithoutChain: Omit<AuditEvent, 'seq' | 'prevHash' | 'hash'>) {
  const s = store();
  for (let attempt = 0; attempt < 5; attempt++) {
    const head = (await s.get('chain/head', { type: 'json' })) as HeadRecord | null;
    const seq = (head?.seq ?? 0) + 1;
    const prevHash = head?.hash ?? null;
    const full = { ...eventWithoutChain, seq, prevHash } as Omit<AuditEvent, 'hash'>;
    const hash = sha256(canonical(full));
    const event: AuditEvent = { ...full, hash };
  }
}
```

```

    // Write the immutable event first (unique key – cannot clobber anything).
    await s.setJSON(keyFor(new Date(event.ts), event.id), event);
    // Then advance the head. If someone else advanced it meanwhile, retry.
    const current = (await s.get('chain/head', { type: 'json' })) as HeadRecord | null;
    if ((current?.seq ?? 0) === (head?.seq ?? 0)) {
        await s.setJSON('chain/head', { seq, hash } satisfies HeadRecord);
        return event;
    }
    // else: contention – the event blob is written but the chain moved; loop
    // recomputes seq/hash against the new head on the next pass. To avoid a
    // duplicate event blob, delete the tentative one before retrying:
    await s.delete(keyFor(new Date(event.ts), event.id));
}
throw new Error('audit chain contention');
}

export interface RecordInput {
    req: Request;
    actor: TokenPayload | null;
    action: string;
    severity?: AuditSeverity;
    outcome?: 'success' | 'failure';
    target?: AuditTarget | null;
    changes?: AuditEvent['changes'];
    metadata?: Record<string, unknown>;
}

/**
 * Record one audit event. Best-effort by default: never throws into the caller,
 * so a logging failure can't break the primary action. For `critical` actions
 * you may choose to await + surface failures (fail-closed) – see §9.
 */
export async function recordAudit(input: RecordInput): Promise<void> {
    try {
        const ctx = requestContext(input.req);
        const base: Omit<AuditEvent, 'seq' | 'prevHash' | 'hash'> = {
            id: randomUUID(),
            ts: new Date().toISOString(),
            action: input.action,
            severity: input.severity ?? 'info',
            outcome: input.outcome ?? 'success',
            actor: actorFrom(input.actor),
            target: input.target ?? null,
            changes: input.changes,
            metadata: input.metadata,
            ...ctx,
        };
        await advanceChain(base);
    } catch (err) {
        // Swallow – auditing must not break the action. Surface to platform logs.
        console.error('audit.record failed', input.action, err);
    }
}

```

5.2 Diff helper (field-level changes)

```
// Build a redacted { field: {from,to} } diff for updates.
const REDACT = new Set(['passwordHash', 'password', 'token', 'apiKey']);

export function diff<T extends Record<string, unknown>>(before: T, after: Partial<T>) {
  const changes: Record<string, { from: unknown; to: unknown }> = {};
  for (const k of Object.keys(after)) {
    if (before[k] !== after[k as keyof T]) {
      const redacted = REDACT.has(k);
      changes[k] = {
        from: redacted ? '«redacted»' : before[k],
        to: redacted ? '«redacted»' : (after as Record<string, unknown>)[k],
      };
    }
  }
  return Object.keys(changes).length ? changes : undefined;
}
```

5.3 Integrating at call sites

The pattern: perform the action, then record. Pass the already-verified JWT payload so no extra work is done. Examples using this codebase's existing functions.

Attendees — permanent delete ([netlify/functions/attendees.mts](#)):

```
if (new URL(req.url).searchParams.get('permanent') === '1') {
  const actor = requireOwner(req);
  const victim = (await readAll()).find((a) => a.ticketCode.toLowerCase() === code.toLowerCase());
  const res = await removeOne(code, true);
  await recordAudit({
    req, actor,
    action: 'attendee.deleted',
    severity: 'critical',
    target: { type: 'attendee', id: victim?.id ?? null, label: victim?.name ?? code },
    metadata: { ticketCode: code },
  });
  return res;
}
```

Attendees — bulk delete ([bulkRemove](#)):

```
await recordAudit({
  req, actor,
  action: permanent ? 'attendee.bulk_deleted' : 'attendee.bulk_archived',
  severity: permanent ? 'critical' : 'warning',
  target: null,
  metadata: { requested: codes.length, removed },
});
```

Auth — role change ([netlify/functions/auth.mts](#)):

```
const actor = requirePermission(req, 'team');
const before = await findUserById(userId);
await setRole(userId, role);
await recordAudit({
  req, actor,
  action: 'team.user.role_changed',
  severity: 'critical',
  target: { type: 'user', id: userId, label: before?.email ?? userId },
  changes: { role: { from: before?.role ?? null, to: role } },
});
```

Auth — failed login (record even though there's no actor):

```
const user = await findByEmail(email);
if (!user || !verifyPassword(password, user.passwordHash)) {
  await recordAudit({
    req, actor: null,
    action: 'auth.login.failure',
    severity: 'warning',
    outcome: 'failure',
    target: { type: 'user', id: user?.id ?? null, label: email },
  });
  throw new AuthError(401, 'Invalid email or password');
}
```

Settings — SMS provider switch (`netlify/functions/settings.mts`):

```
const before = await readSettings();
await writeSettings(next);
await recordAudit({
  req, actor,
  action: before.smsProvider !== next.smsProvider ? 'settings.sms_provider_changed' : 'settings.updated',
  severity: 'warning',
  target: { type: 'settings', id: 'event', label: 'Event settings' },
  changes: diff(before, next),
});
```

Guideline: call `recordAudit` **after** the action succeeds (so `success` outcomes are truthful), and for failures pass `outcome: 'failure'` explicitly. Keep call sites to one statement.

6. Query API

6.1 Endpoint

GET /api/audit	-> filtered, paginated events	[permission: audit]
GET /api/audit/verify	-> chain integrity check (owner only)	[owner]

Both are **owner-restricted** by default (see the new `audit` permission in §11). Reading the audit log is itself audited (`audit.viewed`).

6.2 Query parameters

Param	Meaning		
from , to	ISO date range (defaults to last 30 days)		
actor	filter by actor id or email		
action	exact action key or domain prefix (e.g. attendee.)		
severity	info \	warning \	critical
entity	target type (e.g. attendee)		
cursor	opaque pagination cursor (last key seen)		
limit	page size (default 50, max 200)		

6.3 Function skeleton (`netlify/functions/audit.mts`)

```
import type { Context } from '@netlify/functions';
import { getStore } from '@netlify/blobs';
import { AuthError, requirePermission, requireOwner } from '../shared/auth';
import { recordAudit } from '../shared/audit';
import type { AuditEvent } from '../shared/audit-types';

export const config = { path: ['/api/audit', '/api/audit/verify'] };

const store = () => getStore({ name: 'audit', consistency: 'strong' });

export default async (req: Request, _ctx: Context): Promise<Response> => {
  const pathname = new URL(req.url).pathname;
  try {
    if (pathname.endsWith('/verify')) {
      requireOwner(req);
      return json(await verifyChain());
    }
    const actor = requirePermission(req, 'audit');
    const url = new URL(req.url);
    const from = url.searchParams.get('from');
    const to = url.searchParams.get('to');
    const prefixes = dayPrefixes(from, to); // ['events/2026/07/29/', ...]
    const filters = {
      actor: url.searchParams.get('actor'),
      action: url.searchParams.get('action'),
      severity: url.searchParams.get('severity'),
      entity: url.searchParams.get('entity'),
    };
    const limit = Math.min(Number(url.searchParams.get('limit')) || 50, 200);
    const cursor = url.searchParams.get('cursor');

    const s = store();
    const keys: string[] = [];
    for (const prefix of prefixes) {
      const { blobs } = await s.list({ prefix });
      for (const b of blobs) keys.push(b.key);
    }
    keys.sort().reverse(); // newest first
    const start = cursor ? keys.indexOf(cursor) + 1 : 0;

    const page: AuditEvent[] = [];
    let next: string | null = null;
    for (let i = start; i < keys.length && page.length < limit; i++) {
      const ev = (await s.get(keys[i], { type: 'json' })) as AuditEvent | null;
      if (ev && matches(ev, filters)) page.push(ev);
      next = keys[i];
      if (page.length === limit) break;
    }

    // Reading the log is itself an audited event.
    await recordAudit({ req, actor, action: 'audit.viewed', metadata: { ...filters, count: page.length } });
    return json({ events: page, cursor: page.length === limit ? next : null });
  } catch (err) {
    if (err instanceof AuthError) return json({ error: err.message }, err.status);
    console.error('audit function error', err);
    return json({ error: 'Server error' }, 500);
  }
};
```

`matches`, `dayPrefixes`, `verifyChain`, and `json` are small helpers (full versions in Appendix B).

7. Tamper-evidence (hash chain)

7.1 How it works

Each event stores `prevHash` (the previous event's hash) and its own `hash = sha256(canonical(event-without-hash))`. This links events into a chain: altering or deleting any past event breaks every subsequent hash, which `/api/audit/verify` detects.

```
E1.hash ↙
      ↳ E2.prevHash ; E2.hash ↙
                        ↳ E3.prevHash ; E3.hash → ...
```

7.2 Verification

```
async function verifyChain(): Promise<{ ok: boolean; brokenAt?: number; checked: number }> {
  const s = store();
  const { blobs } = await s.list({ prefix: 'events/' });
  const keys = blobs.map((b) => b.key).sort(); // chronological
  let prevHash: string | null = null;
  let checked = 0;
  for (const key of keys) {
    const ev = (await s.get(key, { type: 'json' })) as AuditEvent;
    const { hash, ...rest } = ev;
    if (sha256(canonical(rest)) !== hash) return { ok: false, brokenAt: ev.seq, checked };
    if (ev.prevHash !== prevHash) return { ok: false, brokenAt: ev.seq, checked };
    prevHash = hash;
    checked++;
  }
  return { ok: true, checked };
}
```

7.3 Threat model & limits

- **Detects:** silent edits, insertions, and deletions of events by anyone with store access.
- **Does not prevent:** an attacker with full store credentials from **rewriting the entire chain** from a point forward. To harden, periodically **anchor** the current `head.hash` somewhere out-of-band (email it to the owner nightly, commit it to a private log, or push to a second store/provider). Any divergence from the last anchored hash is proof of tampering.
- If chain contention (§4.3) is a concern and tamper-evidence is not required for v1, ship **without** the shared head: keep per-event `hash` (self-integrity) and drop `seq` / `prevHash`. You can add chaining later.

8. Security & privacy

8.1 Access control

- The query API requires the new `audit` permission, granted to **owner** only by default (managers can be added later).
- Individual event blobs are never exposed by any public route.
- Every read of the log is itself logged (`audit.viewed` / `audit.exported`).

8.2 PII & redaction

- The log **contains PII by nature** (names, emails, phone numbers of both admins and guests). Treat the `audit` store as **confidential**.
- Redact sensitive fields in `changes` / `metadata` (`passwordHash`, tokens, API keys) — see the `REDACT` set.
- Store **facts, not payloads:** `recipientCount`, `channel`, `ticketCode` — never full message text or credentials.
- Consider a shorter retention for `info` events and a longer one for `warning` / `critical` (§10) to minimize PII footprint.

8.3 Integrity

- App code path is **append-only**: no function ever updates or deletes an individual event except the retention job (§10), which deletes **whole day prefixes** older than the window and records a `audit.retention_pruned` event.

9. Failure handling

- Best-effort by default:** `recordAudit` swallows its own errors so a Blobs hiccup can't fail a check-in or a delete. Failures go to Netlify function logs.
- Fail-closed option for `critical` actions:** for the highest-stakes actions (permanent delete, role change), you may prefer to *guarantee* a record. Two patterns:
 - Await + tolerate:** `await recordAudit(...)` before returning — the action already happened, but you learn immediately if logging failed.
 - Log-first:** write a `pending` audit event *before* the mutation, then a `committed` follow-up. Heavier; usually unnecessary at this scale.
- Never** let audit latency dominate: a single extra Blobs write (+ one head update) per action is well within function time budgets.

10. Retention & lifecycle

10.1 Policy (recommended defaults)

Severity	Retention
<code>info</code>	180 days
<code>warning</code>	400 days
<code>critical</code>	400 days (or archive indefinitely)

10.2 Scheduled prune (Netlify Scheduled Function)

```
// netlify/functions/audit-retention.mts
import type { Config } from '@netlify/functions';
import { getStore } from '@netlify/blobs';

export const config: Config = { schedule: '@daily' };

export default async () => {
  const s = getStore({ name: 'audit', consistency: 'strong' });
  const cutoff = Date.now() - 180 * 24 * 60 * 60 * 1000;
  const { blobs } = await s.list({ prefix: 'events/' });
  let pruned = 0;
  for (const b of blobs) {
    const ev = (await s.get(b.key, { type: 'json' })) as { ts: string; severity: string } | null;
    if (!ev) continue;
    const keep = ev.severity === 'info' ? cutoff : Date.now() - 400 * 24 * 60 * 60 * 1000;
    if (Date.parse(ev.ts) < keep) { await s.delete(b.key); pruned++; }
  }
  console.log('audit retention pruned', pruned);
};
```

Pruning breaks the hash chain by design. Record the pre-prune `head.hash` alongside the prune event, or re-anchor after pruning, so verification can resume from the new baseline.

10.3 Scaling optimization (only if needed)

If event volume ever outgrows per-blob listing, add a **daily rollup**: a scheduled job concatenates each finished day's events into a single `rollups/<YYYY-MM-DD>.ndjson` blob and deletes the per-event blobs. Queries then read one blob per day. Not necessary at current scale.

11. Permission model change

Add an `audit` permission mirroring the existing model (`netlify/shared/auth.ts` and `src/app/admin/services/permissions.ts`):

```
export type Permission =
  | 'dashboard' | 'attendees' | 'reservations' | 'register'
  | 'tickets' | 'checkin' | 'team' | 'settings' | 'sponsors'
  | 'audit'; // NEW

export const ROLE_PERMISSIONS: Record<Role, Permission[]> = {
  owner: [ /* ...all... */, 'audit' ], // owner only, initially
  manager: [ /* ...unchanged... */ ],
  coordinator: [ /* ...unchanged... */ ],
  usher: [ /* ...unchanged... */ ],
};
```

Keep the two copies in sync (the server is the source of truth; the client copy only hides UI).

12. Admin UI

A new **Audit Log** page under the back office (owner-only), following the existing admin page conventions (`adm-*` classes, `adm-icon` , signals).

- **Route:** `/admin/audit` guarded by `permissionGuard('audit')` ; add to `admin.routes.ts` , the sidebar nav, headings map, and a new `audit` icon.
- **Filters row:** date range, actor, action (grouped by domain), severity — mirroring the query params.
- **Table:** time · actor · action (with severity chip) · target · outcome; newest first; cursor pagination ("Load more").
- **Detail drawer:** click a row to see full `changes` , `metadata` , IP, user-agent, request id, and chain fields.
- **Export:** CSV of the current filter (records an `audit.exported` event) — reuse the export approach already used on the Attendees page.
- **Integrity banner:** an owner-only "Verify integrity" button that calls `/api/audit/verify` and shows ✓ *"Chain intact — N events verified"* or ✗ *"Tampering detected at seq X."*

Client service (`src/app/admin/services/audit.service.ts`) mirrors `partner-admin.service.ts` : a `query(params)` returning `{ events, cursor }` , plus `verify()` .

13. Implementation plan (phased)

Phase 1 — Foundation (½ day)

1. Add `netlify/shared/audit-types.ts` and `netlify/shared/audit.ts` (recorder + diff + chain).
2. Add the `audit` permission to server `auth.ts` and client `permissions.ts` .
3. Unit-test `canonical` , `sha256` , `diff` , and a record→read round-trip.

Phase 2 — Instrument mutations (1 day)

1. Add `recordAudit(...)` calls to: auth (login success/failure, role change, deactivate, password reset), attendees (register, check-in, override, archive, bulk, restore, delete, bulk-delete, email, sms, export), settings, partners, reservations, sms broadcast.
2. Verify events land in the store (temporary debug list).

Phase 3 — Query API + verify (½ day)

1. Add `netlify/functions/audit.mts` (query + verify) with pagination and filters.

Phase 4 — Admin UI (1 day)

1. `audit.service.ts` , the Audit Log page, route, sidebar nav, `audit` icon, detail drawer, CSV export, integrity banner.

Phase 5 — Lifecycle & hardening (½ day)

1. `audit-retention.mts` scheduled prune.
2. Nightly hash anchor (email owner the current `head.hash`) — optional.
3. Docs: update `CLAUDE.md` /README with the audit action catalog.

Total: ~3.5 developer-days. Phases 1–3 deliver a working, queryable trail; 4–5 add UX and lifecycle.

14. Testing strategy

- **Unit:** canonical JSON stability, hash determinism, `diff` redaction, retention cutoff math.
- **Integration:** call each instrumented endpoint and assert exactly one event with the right `action`, `actor`, `target`, `outcome`.
- **Concurrency:** fire N parallel deletes and assert N distinct event blobs exist (no lost updates) and the chain verifies.
- **Tamper test:** mutate one stored event; assert `/api/audit/verify` reports `ok:false` at the right `seq`.
- **Authz:** non-owner gets 403 from `/api/audit`; anonymous gets 401.
- **PII:** assert no `passwordHash` /token appears in any recorded event.

15. Risks & mitigations

Risk	Mitigation
Audit write fails and silently loses a record	Best-effort + platform-log the failure; fail-closed option for critical actions (§9)
Chain head contention under concurrency	Optimistic retry (§4.3); or ship without shared head (§7.3)
Log grows unbounded / cost	Date-prefixed keys + scheduled retention (§10); rollup if needed
Log leaks PII	Owner-only access, redaction, store facts-not-payloads, shorter <code>info</code> retention (§8)
Full-store compromise rewrites history	Out-of-band hash anchoring (§7.3)
Two permission copies drift	Server is source of truth; add a test asserting parity

Appendix A — Full action catalog

See the table in §2. Each entry maps to exactly one `action` key. Add new keys by domain prefix (`<domain>.<verb>[.<qualifier>]`) and give each a severity.

Appendix B — Helper implementations

```
// dayPrefixes: enumerate 'events/YYYY/MM/DD/' between from..to (inclusive),
// defaulting to the last 30 days when unset.
export function dayPrefixes(from?: string | null, to?: string | null): string[] {
  const end = to ? new Date(to) : new Date();
  const start = from ? new Date(from) : new Date(end.getTime() - 30 * 864e5);
  const out: string[] = [];
  for (let d = new Date(Date.UTC(start.getUTCFullYear(), start.getUTCMonth(), start.getUTCDate()));
    d <= end; d.setUTCDate(d.getUTCDate() + 1)) {
    const y = d.getUTCFullYear(), m = String(d.getUTCMonth() + 1).padStart(2, '0'),
      da = String(d.getUTCDate()).padStart(2, '0');
    out.push(`events/${y}/${m}/${da}/`);
  }
  return out;
}

export function matches(ev: AuditEvent, f: Record<string, string | null>): boolean {
  if (f.actor && ev.actor.id !== f.actor && ev.actor.email !== f.actor) return false;
  if (f.action && !ev.action.startsWith(f.action)) return false;
  if (f.severity && ev.severity !== f.severity) return false;
  if (f.entity && ev.target?.type !== f.entity) return false;
  return true;
}

function json(body: unknown, status = 200): Response {
  return new Response(JSON.stringify(body), {
    status,
    headers: { 'Content-Type': 'application/json' },
  });
}
```

Appendix C — Example event

```
{
  "id": "6f1c9e2a-1b7d-4a3e-9c11-2f0c8f4b9a10",
  "seq": 1487,
  "ts": "2026-07-30T18:04:12.481Z",
  "action": "attendee.bulk_deleted",
  "severity": "critical",
  "outcome": "success",
  "actor": { "id": "u_01H...", "name": "John Mgbako", "email": "jmgbako@gmail.com", "role": "owner" },
  "target": null,
  "metadata": { "requested": 12, "removed": 12 },
  "ip": "102.88.54.144",
  "userAgent": "Mozilla/5.0 ...",
  "requestId": "01J...",
  "prevHash": "9b1c...",
  "hash": "c4f8..."
}
```